



netweevil QGIS Plugin

This plugin talks directly to the running `netweevil` API.

It does not shell out to the CLI. QGIS sends JSON requests to the API, receives JSON or GeoJSON responses, and loads the returned geometries into the map immediately.

It runs on QGIS 3.28+ (Qt 5) and QGIS 4 (Qt 6). The `supportsQt6` metadata flag is set and all Qt/QGIS enum differences are handled in `netweevil_qgis/compat.py`.

What It Does

- connects to a running `netweevil api` serve instance, automatically on open and lazily before every run
- reads the loaded dataset and available profiles from `/v1/service`
- lets you pick route start and end points directly from the QGIS map canvas
- can pull route points from a selected point feature in the active layer
- submits route, OD, matrix, and service-area requests to the API
- submits scheduled transit route requests when the API has preloaded GTFS feeds
- runs multi-mode agent-based traffic simulations with mid-run control, animated playback, congestion and busy-segment layers
- loads route, transit, OD, matrix, and service-area geometries into QGIS as grouped layers
- reloads completed JSON/GeoJSON runs and succeeded run manifests without rerunning the API request
- can build matrix origins and destinations from loaded QGIS point layers
- can build service-area origins from map picks, selected point features, or the current route endpoints
- exposes disconnected-network policies and opt-in degraded-routing controls for route and batch requests
- remembers connection and analysis settings between QGIS sessions — except unsafe degraded-routing modes, which always start off

Layout

The dock has a persistent connection bar at the top (API URL, Connect button, status indicator, and the active routing profile), followed by task tabs:

Tab	Purpose
Route	Single road-network route between two picked points
Transit	Pedestrian access + scheduled GTFS transit route
Service Area	Reachable network/polygon areas around one or more origins
OD / Matrix	Batch origin-destination pairs and full travel matrices
Simulation Runs	Agent-based traffic simulation with live playback Reload previously saved outputs without re-running
Settings	Workspace root, timeout, saved-file format, dataset info

Each tab puts the primary workflow (points, origins, inputs) first; advanced options live in collapsible sections. Rarely needed unsafe degraded-routing modes are collapsed, off by default, never persisted across sessions, highlighted in red while enabled, and confirmed before any request is sent.

Install In QGIS

1. Start the API:

```
cargo run -p netweevil-cli -- api serve \
  --dataset groningen_2026_05 \
  --default-profile examples/profiles/car_research_v1.yml \
  --profile examples/profiles/pedestrian_research_v1.yml \
  --transit-feed openov_groningen \
  --bind 127.0.0.1:8080
```

Omit `--transit-feed` if you only need road-network analyses.

2. Install the plugin, either:

- from a packaged ZIP: Plugins → Manage and Install Plugins → Install from ZIP with `dist/netweevil_qgis-<version>.zip`, or
- by symlinking the source for development:

macOS, QGIS 3

```
ln -s /path/to/netweevil/qgis_plugin/netweevil_qgis \
  ~/Library/Application\
  ↳ Support/QGIS/QGIS3/profiles/default/python/plugins/netweevil_qgis
```

macOS, QGIS 4

```
ln -s /path/to/netweevil/qgis_plugin/netweevil_qgis \
  ~/Library/Application\
  ↳ Support/QGIS/QGIS4/profiles/default/python/plugins/netweevil_qgis
```

3. Enable the plugin from Plugins → Manage and Install Plugins.

Package ZIP

Build a QGIS-installable ZIP from the repository root:

```
./scripts/package_qgis_plugin.sh
```

The script reads version from `qgis_plugin/netweevil_qgis/metadata.txt`, includes every module in the `netweevil_qgis` package, and writes:

```
dist/netweevil_qgis-<version>.zip
```

First Run

1. Open the dock from the netweevil toolbar button.
2. The plugin tries to connect to `http://127.0.0.1:8080` automatically. If your API runs elsewhere, change the URL in the connection bar and press Connect (or just press Run on any tab — the plugin reconnects lazily).
3. The status line shows the connected dataset, profile count, and transit feed count. Pick a profile in the connection bar, or keep the service default.
4. Optionally set Workspace root in the Settings tab. It is only used to resolve relative request/response paths such as `.netweevil/runs/...`; pointing it at the folder where the API runs keeps QGIS outputs next to the rest of the workspace. It defaults to your home folder.

Example Flows

Route

1. In the Route tab, press Pick On Map under Start and click the map; same for End (clicking the button again cancels picking).
2. Optionally use From Selected Feature if you have a point selected in the active point layer.
3. Press Run Route.

The plugin loads the route into a grouped set of QGIS layers and tables: main route, segment rows, hop segments, violations, and road-type/surface breakdowns when the API returns them. The route id auto-advances after each run so results are never overwritten. The Returned Route Detail and Connectivity + Fallback sections hold the advanced request flags.

Transit

Start the API with one or more `--transit-feed <feed_id>` values, then:

1. In the Transit tab, choose a loaded feed and a departure time such as `2026-05-11T08:30:00+02:00`.
2. Pick an origin and destination on the map, or use selected point features.
3. Optionally adjust allowed modes and access/transfer limits under Modes and Limits.
4. Press Run Transit Route.

The plugin saves the raw JSON response and loads a grouped route summary plus per-leg, stop, and stop-segment layers. Access, egress, transfer, and transit legs are styled separately.

Service Area

1. In the Service Area tab, append origins with Pick On Map, Add Selected Features, Use Route Start, or Use Route Start + End.
2. Enter a threshold list such as 300, 600 and pick its unit (Distance (m) or Travel time (s)).
3. Choose road-network options (output/band/boundary/multi-origin modes) or switch Mode to Transit feed — the tab shows only the controls the selected mode actually sends.
4. Press Run Service Area.

Outputs load as grouped sublayers by threshold and geometry type.

OD / Matrix

- OD: point Pairs file at a CSV (id, source_lon, source_lat, target_lon, target_lat, several aliases accepted) or JSON pairs document, then press Run OD.
- Matrix: build origins/destinations from loaded QGIS point layers (optionally selected features only, with an ID field) or from CSV/JSON files, then press Run Matrix.

Simulation

1. Add one or more fleets (profile, agent count, origin/destination distributions, departure curve, behavior).
2. Optionally add polygon zones (no-access, speed/capacity factors, high traffic, spawn/attract).
3. Press Run Simulation, then control it live (pause/resume/cancel, global speed factor, spawn fleets, add/remove zones mid-run).
4. Use the playback slider or Play for animated agents, and load congestion windows, busy segments, or a temporal layer for the QGIS Temporal Controller.

Saved Runs

Use the Runs tab to reload previous outputs without re-running: pick a file from the runs directory listing (or browse to one), leave Kind on Auto detect, and press Load Run. The loader handles API response JSON, direct GeoJSON, raw CLI result JSON, and succeeded run manifests that point to a result file.

Notes

- The plugin does not embed the Rust engine; it calls the running HTTP API.
- Relative request/input/output paths are resolved against the configured workspace root.
- OD and matrix CSV inputs are parsed locally and sent to the API as JSON.
- Picked points and layer features are transformed from the map/layer CRS into WGS84 before being sent to the API.
- Route and transit requests always use JSON responses internally so segment rows, breakdown tables, and itinerary legs stay available; the Saved OD/Matrix/Area format setting in the Settings tab only affects how those batch responses are written to disk.
- Connection checks use a short timeout so an unreachable API never freezes QGIS for the full analysis timeout.
- All analysis options are sent as explicit request fields; saved responses keep dataset/profile hashes and run metadata for provenance.